WEEK 4

# Android Basics: Your First Compose App

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

This week you move from the Kotlin Playground to **real Android apps**. You'll set up Android Studio, understand a project's files, and build screens with **Jetpack Compose**: text, modifiers, buttons, icons and images. At the end of the week you'll build a **Business Card app** and run it on your phone.

| Day | Topic | You will be able to... |
|---|---|---|
| Day 1 | Android Studio & your first app | Create, understand and run an Android project |
| Day 2 | Composable functions & Text | Write @Composable functions and style text |
| Day 3 | Modifiers | Control size, spacing, background, shape and clicks |
| Day 4 | Buttons, icons & images | Add interactive buttons, icons and pictures |
| Day 5 | First layouts, Surface & Card | Combine elements with Column, Row and Spacer |
| Day 6 | Weekly project | Build and run a Business Card app |

## Before you start

- Your computer needs **8 GB RAM** (16 GB is better) and about 15 GB of free disk space.
- Create one practice project, **Week4Practice** (Empty Activity), and try every example in it.
- Look at each example in the **Preview** panel (Split view). Run it on a device at least once a day.

**Tip:** Every Compose example in this book was **compile-checked** with the Compose compiler. **Import lines are left out** to keep the code short: when a word turns red, click it and press **Alt + Enter** → **Import**. Code marked **Android only** (blue background) is shown for reference.

WEEK 4 • DAY 1

# Android Studio & Your First App

**Today's goal:** Install Android Studio, create a Compose project, understand its files and run it on an emulator or phone.

## 1. Install Android Studio

- Download it free from **developer.android.com/studio** and install it with the **Standard** setup.
- The first launch downloads the Android SDK. Let it finish completely (it can take a while).
- Keep Android Studio updated: **Help** → **Check for Updates** (on a Mac, **Android Studio** → **Check for Updates**).

## 2. Create a new project

| Step | What to choose |
|---|---|
| New Project | Phone and Tablet → **Empty Activity** (the one with the Compose logo) |
| Name | Week4Practice |
| Package name | com.example.week4practice (fine for practice) |
| Minimum SDK | API 24 (Android 7.0): runs on almost every phone |
| Build configuration language | Kotlin DSL (build.gradle.kts) |

Click **Finish** and wait until **Gradle sync** finishes (the progress bar at the bottom disappears). The first sync can take 5 to 15 minutes.

## 3. The project structure

**Sketch: The Android view of the project panel**

```
app
 |-- manifests/
 |    +-- AndroidManifest.xml        <- app name, icon, permissions, which screen opens first
 |-- kotlin+java/
 |    +-- com.example.week4practice/
 |         |-- MainActivity.kt       <- the app's entry point; your code starts here
 |         +-- ui.theme/             <- Color.kt, Theme.kt, Type.kt
 +-- res/                            <- resources
      |-- drawable/                  <- images and icons
      |-- mipmap/                    <- the app launcher icon
      +-- values/                    <- strings.xml, colors.xml, themes.xml
Gradle Scripts
 |-- build.gradle.kts (Module :app)  <- app settings and libraries (dependencies)
 +-- libs.versions.toml              <- library version numbers
```

## 4. Gradle in two minutes

**Gradle** is the tool that builds your app: it compiles Kotlin, packages resources and creates the APK. The two files you'll look at most are:

| File | What's inside |
|---|---|
| build.gradle.kts (Module :app) | compileSdk, minSdk, targetSdk, and the **dependencies** block listing the libraries the app uses |

| File | What's inside |
|---|---|
| libs.versions.toml | The version numbers of every library, in one place |

**Careful:** Don't upgrade library versions just because Android Studio highlights them in yellow. Newer libraries can require a newer Android Studio, Gradle plugin and compileSdk (remember the **AAR metadata** error?). In a team project, **never** change versions without asking.

## 5. MainActivity: where the app starts

**MainActivity.kt as created by the template (Android only)**

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Week4PracticeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(text = "Hello $name!", modifier = modifier)
}
```

| Code | Meaning (with the week you learned it) |
|---|---|
| class MainActivity : ComponentActivity() | MainActivity **inherits** from ComponentActivity (Week 3, Day 3) |
| override fun onCreate(...) | Android calls this when the screen is created. We **override** it (Week 3) |
| super.onCreate(...) | Run the parent's setup first (Week 3, super) |
| setContent { ... } | "Here is my UI." A **trailing lambda** (Week 2, Day 4) |
| Week4PracticeTheme { } | Applies the app's colours and fonts to everything inside |
| Scaffold { innerPadding -> } | A basic screen frame. innerPadding keeps content clear of the status bar |
| modifier: Modifier = Modifier | A **default argument** (Week 1, Day 6) |

## 6. Run your app

| Option | Steps |
|---|---|
| Emulator (virtual phone) | **Device Manager** (phone icon on the right) → **+** → Create Virtual Device → choose a Pixel phone → download a system image → Finish. Select it in the toolbar and press **Run** |
| Your phone (often faster) | Settings → About phone → tap **Build number** 7 times. Then Developer options → turn on **USB debugging**. Connect it with a cable, tap Allow, select it in the toolbar and press **Run** |

## 7. Logcat: println for Android

Android apps don't show println output on screen. Use **Log.d** and read it in the **Logcat** window at the bottom of Android Studio.

**Logging (Android only)**

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d("Week4", "onCreate called")          // tag, message
    setContent { /* ... */ }
}
```

In Logcat, type **tag:Week4** in the search box to see only your messages. If the app crashes, look for red text with **FATAL EXCEPTION**. The lines below it show the cause.

## Practice Exercises

**1.** Create the Week4Practice project and run it on an emulator **and** on your phone (if you have a cable).

**2.** Change "Android" to your name, run it again, and add a Log.d message in onCreate. Find it in Logcat.

**3.** Open AndroidManifest.xml and find the line that says which Activity starts first (look for LAUNCHER).

**4.** Open build.gradle.kts (Module :app) and write down the minSdk, targetSdk and compileSdk values.

*Solutions are in the Answer Key at the end. Try first and look later!*

## Quick Quiz

**Q1.** Which template should you choose for a Compose app?

**Q2.** What is Gradle?

**Q3.** Where do images go in a project?

**Q4.** Who calls onCreate()?

**Q5.** How do you print debug messages in an Android app?

WEEK 4 • DAY 2

# Composable Functions & Text

**Today's goal:** Write @Composable functions, use Preview, and display and style text.

## 1. What is a composable function?

In Jetpack Compose, UI is built from **functions** marked with **@Composable**. Each one **describes** a piece of the screen. You don't create views and change them later. You describe what the screen should look like **for the current data**.

**A composable and its preview**

```
@Composable
fun WelcomeMessage(name: String) {
    Text(text = "Welcome to Android, $name!")
}

@Preview(showBackground = true)
@Composable
fun WelcomeMessagePreview() {
    WelcomeMessage(name = "Anjana")
}
```

| Rule | Example |
|---|---|
| Mark the function with @Composable | @Composable fun WelcomeMessage(...) |
| Name it like a noun, starting with a capital letter | WelcomeMessage, ProfileCard (not showMessage) |
| It returns nothing (Unit). It emits UI instead | No return type is written |
| Call composables only from other composables | setContent { }, or another @Composable function |
| Pass the data it shows as parameters | name: String |

## 2. Preview

**@Preview** shows a composable in the Preview panel without running the app. Click **Split** at the top right of the editor. A preview function must have **no parameters**, so it calls your composable with sample data.

**Preview options**

```
@Preview(name = "Short name", showBackground = true)
@Preview(name = "Long name", showBackground = true, widthDp = 200)
@Composable
fun WelcomeMessagePreviews() {
    Text(text = "Welcome to Android, Anjana!")
}
```

**Tip:** You can stack several @Preview annotations on one function to see different sizes side by side. **widthDp = 200** simulates a narrow screen.

## 3. Styling text

**Text parameters**

```
@Composable
fun StyledTexts() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = "Big and bold", fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(text = "Italic", fontStyle = FontStyle.Italic)
        Text(text = "Primary colour", color = MaterialTheme.colorScheme.primary)
        Text(text = "Underlined", textDecoration = TextDecoration.Underline)
        Text(text = "Wide letters", letterSpacing = 4.sp)
        Text(
            text = "Centred across the full width",
            textAlign = TextAlign.Center,
            modifier = Modifier.fillMaxWidth()
        )
        Text(
            text = "This sentence is far too long to fit on one line of a phone screen",
            maxLines = 1,
            overflow = TextOverflow.Ellipsis
        )
    }
}
```

| Parameter | Example values |
|---|---|
| fontSize | 16.sp, 24.sp (always **sp** for text) |
| fontWeight | FontWeight.Normal, Medium, SemiBold, Bold |
| color | MaterialTheme.colorScheme.primary (avoid fixed colours) |
| textAlign | TextAlign.Start, Center, End (needs width, e.g. fillMaxWidth) |
| maxLines + overflow | maxLines = 2, overflow = TextOverflow.Ellipsis (shows "...") |

## 4. Typography styles: the professional way

Instead of setting fontSize and fontWeight everywhere, use the theme's ready-made **typography styles**. Then the whole app looks consistent, and a designer can change all fonts in one place.

**MaterialTheme.typography**

```
@Composable
fun TypographyExamples() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Display Small", style = MaterialTheme.typography.displaySmall)
        Text("Headline Medium", style = MaterialTheme.typography.headlineMedium)
        Text("Title Large", style = MaterialTheme.typography.titleLarge)
        Text("Title Medium", style = MaterialTheme.typography.titleMedium)
        Text("Body Large: normal reading text", style = MaterialTheme.typography.bodyLarge)
        Text("Body Small: less important text", style = MaterialTheme.typography.bodySmall)
        Text("LABEL MEDIUM", style = MaterialTheme.typography.labelMedium)
    }
}
```

## 5. Composables calling composables

**Building a small piece of UI from smaller pieces**

```
@Composable
fun SectionTitle(text: String) {
    Text(text = text, style = MaterialTheme.typography.titleMedium)
}

@Composable
fun InfoLine(label: String, value: String) {
    Text(text = "$label: $value", style = MaterialTheme.typography.bodyLarge)
}

@Composable
fun StudentInfo(name: String, course: String, city: String = "Kochi") {
    Column(modifier = Modifier.padding(16.dp)) {
        SectionTitle("Student")
        InfoLine("Name", name)
        InfoLine("Course", course)
        InfoLine("City", city)
    }
}

@Preview(showBackground = true)
@Composable
fun StudentInfoPreview() {
    StudentInfo(name = "Anjana", course = "Android Development")
}
```

**Column** places its children one below another. You'll learn layouts properly on Day 5 and in Week 5. For now, use it to stack several Texts.

## 6. Logic inside composables

A composable is a normal Kotlin function, so you can use **if**, **when** and loops inside it to decide what to show.

**if, when and loops in UI**

```
@Composable
fun ResultCard(name: String, marks: Int, subjects: List<String>) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(name, style = MaterialTheme.typography.titleLarge)

        if (marks >= 40) {
            Text("Passed", color = MaterialTheme.colorScheme.primary)
        } else {
            Text("Failed", color = MaterialTheme.colorScheme.error)
        }

        val grade = when {
            marks >= 90 -> "A+"
            marks >= 75 -> "A"
            marks >= 60 -> "B"
            else -> "C"
        }
        Text("Grade: $grade")

        for (subject in subjects) {
            Text("- $subject", style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

## Practice Exercises

**1.** Write a composable **MovieTitle(title, year)** that shows the title in titleLarge and the year in bodyMedium. Add a preview.

**2.** Write **QuoteCard(quote, author)**: the quote in italics and the author right-aligned across the full width.

**3.** Write **WeatherInfo(city, temperature: Int)** that shows "Hot" in the error colour if the temperature is above 35, otherwise "Pleasant".

**4.** Write **ShoppingList(items: List<String>)** that shows a title and then each item on its own line using a for loop.

*Solutions are in the Answer Key at the end. Try first and look later!*

## Quick Quiz

**Q1.** What does @Composable mean?

**Q2.** Why does a preview function have no parameters?

**Q3.** Which unit is used for text size?

**Q4.** Why use MaterialTheme.typography instead of fontSize?

**Q5.** Can you use if and for inside a composable?

WEEK 4 • DAY 3

# Modifiers

**Today's goal:** Use modifiers to control size, spacing, background, borders, shape and clicks.

## 1. What is a Modifier?

A **Modifier** changes how a composable looks or behaves: its size, padding, background, shape or click behaviour. You chain modifiers with dots, and they are applied **in order**.

**Your first modifiers**

```
@Composable
fun ModifierBasics() {
    Text(
        text = "Hello Modifiers",
        modifier = Modifier
            .padding(16.dp)                                    // space around
            .background(MaterialTheme.colorScheme.primaryContainer)
            .padding(12.dp)                                    // space inside the colour
    )
}
```

## 2. Size modifiers

**Width, height and size**

```
@Composable
fun SizeExamples() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("fillMaxWidth", modifier = Modifier
            .fillMaxWidth()
            .background(MaterialTheme.colorScheme.secondaryContainer))
        Text("fillMaxWidth(0.5f): half", modifier = Modifier
            .fillMaxWidth(0.5f)
            .background(MaterialTheme.colorScheme.tertiaryContainer))
        Text("width(150.dp)", modifier = Modifier
            .width(150.dp)
            .background(MaterialTheme.colorScheme.secondaryContainer))
        Box(modifier = Modifier
            .size(64.dp)                    // same width and height
            .background(MaterialTheme.colorScheme.primary))
        Text("height(48.dp)", modifier = Modifier
            .height(48.dp)
            .background(MaterialTheme.colorScheme.tertiaryContainer))
    }
}
```

| Modifier | Meaning |
|---|---|
| fillMaxWidth() | As wide as the parent allows |
| fillMaxSize() | As wide and as tall as the parent allows |
| width(x.dp) / height(x.dp) / size(x.dp) | An exact size |
| (none) | By default, a composable is as small as its content ("wrap content") |

## 3. Padding

**Different kinds of padding**

```
@Composable
fun PaddingExamples() {
    Column {
        Text("All sides 16.dp", modifier = Modifier.padding(16.dp))
        Text("Left/right 24, top/bottom 8",
            modifier = Modifier.padding(horizontal = 24.dp, vertical = 8.dp))
        Text("Only the start (left) 32",
            modifier = Modifier.padding(start = 32.dp))
        Text("Each side different",
            modifier = Modifier.padding(start = 8.dp, top = 4.dp, end = 8.dp, bottom = 16.dp))
    }
}
```

**Tip:** Use **start** and **end** instead of left and right. They flip automatically for right-to-left languages.

## 4. Order matters!

**background then padding vs padding then background**

```
@Composable
fun OrderMatters() {
    Row(modifier = Modifier.padding(16.dp)) {
        Text("A", modifier = Modifier
            .background(MaterialTheme.colorScheme.primaryContainer)
            .padding(20.dp))
        Spacer(Modifier.width(24.dp))
        Text("B", modifier = Modifier
            .padding(20.dp)
            .background(MaterialTheme.colorScheme.primaryContainer))
    }
}
```

**Sketch: Result**




Each modifier wraps everything after it. For A, the background is drawn first and the padding is inside the colour. For B, the padding comes first, so the colour only covers the text.

## 5. Shape, border and clip

**Rounded corners and circles**

```
@Composable
fun ShapeExamples() {
    Row(modifier = Modifier.padding(16.dp), horizontalArrangement = Arrangement.spacedBy(16.dp)) {
        Box(modifier = Modifier
            .size(64.dp)
            .clip(RoundedCornerShape(12.dp))
            .background(MaterialTheme.colorScheme.primary))
        Box(modifier = Modifier
            .size(64.dp)
            .clip(CircleShape)
            .background(MaterialTheme.colorScheme.secondary))
        Box(modifier = Modifier
            .size(64.dp)
            .border(2.dp, MaterialTheme.colorScheme.outline, RoundedCornerShape(12.dp)))
    }
}
```

## 6. Making things clickable

**clickable**

```
@Composable
fun ClickableTag(label: String, onClick: () -> Unit) {
    Text(
        text = label,
        modifier = Modifier
            .clip(RoundedCornerShape(50))                    // pill shape
            .background(MaterialTheme.colorScheme.secondaryContainer)
            .clickable { onClick() }                         // ripple on tap
            .padding(horizontal = 16.dp, vertical = 8.dp)
    )
}
```

**onClick: () -> Unit** is a function-type parameter (Week 2, Day 4). The tag doesn't decide what happens on a click: its **caller** does. Put **clip** before **clickable** so the ripple effect follows the rounded shape.

## 7. The modifier parameter rule

**Accept a modifier, apply it to the root**

```
@Composable
fun Badge(text: String, modifier: Modifier = Modifier) {
    Text(
        text = text,
        style = MaterialTheme.typography.labelSmall,
        modifier = modifier                                  // the caller's modifier first
            .background(MaterialTheme.colorScheme.error, RoundedCornerShape(8.dp))
            .padding(horizontal = 8.dp, vertical = 2.dp),
        color = MaterialTheme.colorScheme.onError
    )
}

@Composable
fun BadgeUsage() {
    Badge(text = "NEW", modifier = Modifier.padding(16.dp))   // the caller adds spacing
}
```

> **Tip:** Every reusable composable should have **modifier: Modifier = Modifier** as its first optional parameter, applied to its outermost element. This is Android's official guideline, and every team expects it.

## Practice Exercises

**1.** Create a 100.dp square Box with rounded corners (16.dp), a primary background and a 2.dp outline border.

**2.** Recreate examples A and B from section 4, then try putting clip after background. What changes?

**3.** Make a **Chip(label, selected: Boolean, onClick)** that uses primaryContainer when selected and surfaceVariant when not.

**4.** Create a full-width banner Text with 24.dp padding, centred text and a tertiaryContainer background.

*Solutions are in the Answer Key at the end. Try first and look later!*

## Quick Quiz

**Q1.** In what order are modifiers applied?

**Q2.** What is the difference between fillMaxWidth() and width(200.dp)?

**Q3.** Why put clip before clickable?

**Q4.** Why use start/end instead of left/right?

**Q5.** What should a reusable composable's modifier parameter look like?

WEEK 4 • DAY 4

# Buttons, Icons & Images

**Today's goal:** Add buttons that respond to taps, show Material icons, and display images from resources.

## 1. Buttons

**Button types**

```
@Composable
fun ButtonTypes(onAction: (String) -> Unit) {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Button(onClick = { onAction("Save") }) { Text("Save") }
        FilledTonalButton(onClick = { onAction("Add to cart") }) { Text("Add to cart") }
        OutlinedButton(onClick = { onAction("Cancel") }) { Text("Cancel") }
        TextButton(onClick = { onAction("Skip") }) { Text("Skip") }
        ElevatedButton(onClick = { onAction("Share") }) { Text("Share") }
        Button(onClick = { }, enabled = false) { Text("Disabled") }
    }
}
```

Notice that the button's label is **not** a text parameter: it's a **trailing lambda** containing a Text. That's why you can put anything inside a button, including an icon and text together.

| Button | Use for |
|---|---|
| Button (filled) | The **main** action on the screen. Usually only one |
| FilledTonalButton | Important secondary actions |
| OutlinedButton | Secondary actions, like Cancel next to Save |
| TextButton | Low-priority actions: Skip, Forgot password?, dialog buttons |
| ElevatedButton | A button that needs to stand out from a coloured background |

## 2. What happens on click?

**Responding to a click (Android only)**

```
val context = LocalContext.current

Button(onClick = {
    Log.d("Week4", "Button clicked")                              // shows in Logcat
    Toast.makeText(context, "Saved!", Toast.LENGTH_SHORT).show()  // a small popup message
}) {
    Text("Save")
}
```

To **change what's on the screen** when a button is clicked (like a counter), you need **state**. That's Week 6. This week, use Log.d or a Toast to check that your clicks work.

## 3. Buttons with icons

**Icon + text in a button**

```
@Composable
fun IconButtons() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        Button(onClick = { }) {
            Icon(Icons.Default.ShoppingCart, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Add to cart")
        }
        Row {
            IconButton(onClick = { }) {
                Icon(Icons.Default.FavoriteBorder, contentDescription = "Add to favourites")
            }
            IconButton(onClick = { }) {
                Icon(Icons.Default.Share, contentDescription = "Share")
            }
            IconButton(onClick = { }) {
                Icon(Icons.Default.MoreVert, contentDescription = "More options")
            }
        }
        FloatingActionButton(onClick = { }) {
            Icon(Icons.Default.Add, contentDescription = "Add item")
        }
    }
}
```

## 4. Icons

**Icon styles, sizes and colours**

```
@Composable
fun IconExamples() {
    Row(
        modifier = Modifier.padding(16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(Icons.Filled.Favorite, contentDescription = "Liked")
        Icon(Icons.Outlined.FavoriteBorder, contentDescription = "Not liked")
        Icon(
            imageVector = Icons.Default.Star,
            contentDescription = "Rating",
            tint = MaterialTheme.colorScheme.primary,
            modifier = Modifier.size(40.dp)
        )
        Icon(Icons.Default.Settings, contentDescription = "Settings",
            tint = MaterialTheme.colorScheme.outline)
    }
}
```

| Where icons come from | Notes |
|---|---|
| Icons.Default / Icons.Filled | Filled style. Common icons: Add, Home, Search, Person, Settings, Star, Favorite, Delete, Share... |
| Icons.Outlined / Rounded / Sharp | Other styles of the same icons |
| material-icons-extended library | Hundreds more (Visibility, CameraAlt...). Add the dependency if you need them |
| res/drawable (Vector Asset) | Right-click res → New → Vector Asset to import any Material icon or SVG |

> **Tip:** Give every **tappable** icon a **contentDescription** (TalkBack reads it aloud for blind users). Use **null** for decorative icons that sit next to text saying the same thing.

## 5. Adding an image to your project

- Find a PNG or JPG on your computer, for example a photo called **profile_photo.jpg**.
- Copy it, then in Android Studio right-click **res** → **drawable** and paste.
- The file name must be **lowercase**, with only letters, numbers and underscores: **profile_photo.jpg** works; **Profile Photo.jpg** does not.
- Android creates **R.drawable.profile_photo**, which you use in code.

**Showing an image (Android only)**

```
@Composable
fun ProfilePhoto() {
    Image(
        painter = painterResource(id = R.drawable.profile_photo),
        contentDescription = "Anjana's profile photo",
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .size(120.dp)
            .clip(CircleShape)                                  // a round photo
            .border(3.dp, MaterialTheme.colorScheme.primary, CircleShape)
    )
}

@Composable
fun Banner() {
    Image(
        painter = painterResource(id = R.drawable.banner),
        contentDescription = null,                              // decorative
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .fillMaxWidth()
            .height(180.dp)
    )
}
```

| ContentScale | What it does |
|---|---|
| Crop | Fills the whole area and cuts off the extra parts (best for photos) |
| Fit | Shows the whole image, which may leave empty space |
| FillWidth | Fills the width and keeps the proportions |
| FillBounds | Stretches the image to fill the area (usually looks distorted) |

Images from the **internet** (like user photos) need a library such as **Coil**: `AsyncImage(model = url, ...)`. You'll use that later when you learn networking (Week 10).

## Practice Exercises

**1.** Build a column of 4 buttons: Login (Button), Sign up (OutlinedButton), Continue as guest (TextButton) and a disabled Button.

**2.** Make a row of 4 IconButtons for a music player: previous (Icons.Default.SkipPrevious is extended, so use KeyboardArrowLeft), play (PlayArrow), next (KeyboardArrowRight) and favourite (FavoriteBorder).

**3.** Add a photo to res/drawable and show it as a 100.dp circle with a border. (Android only: try it in your project.)

**4.** Make a Button that shows a Toast saying "Hello Anjana!" when clicked. (Android only.)

*Solutions are in the Answer Key at the end. Try first and look later!*

## Quick Quiz

**Q1.** Why is a button's label a lambda instead of a text parameter?

**Q2.** Which button type should the main action use?

**Q3.** When should contentDescription be null?

**Q4.** What are the rules for image file names in res/drawable?

**Q5.** Which ContentScale is best for profile photos?

WEEK 4 • DAY 5

# First Layouts, Surface & Card

**Today's goal:** Combine elements with Column, Row and Spacer, and group content with Surface and Card.

## 1. Column and Row

**Column** places children **vertically** (one below another). **Row** places them **horizontally** (side by side). Most screens are Columns and Rows inside each other.

**Column and Row together**

```
@Composable
fun ContactLine(name: String, phone: String) {
    Row(
        modifier = Modifier.padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(Icons.Default.Person, contentDescription = null, modifier = Modifier.size(40.dp))
        Spacer(Modifier.width(12.dp))
        Column {
            Text(name, style = MaterialTheme.typography.titleMedium)
            Text(phone, style = MaterialTheme.typography.bodyMedium)
        }
    }
}
```

**Sketch: Structure**

```
Row ------------------------------------------
| [Icon]   Column ------------------        |
|          | Anjana (titleMedium)  |        |
|          | 98470 11111           |        |
|          -------------------------        |
-------------------------------------------
```

## 2. Spacer, alignment and spacing

**Alignment and spacing basics**

```
@Composable
fun CenteredWelcome() {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,   // centre children left-to-right
        verticalArrangement = Arrangement.spacedBy(8.dp)      // 8.dp gap between children
    ) {
        Icon(Icons.Default.Favorite, contentDescription = null,
            tint = MaterialTheme.colorScheme.primary, modifier = Modifier.size(48.dp))
        Text("Welcome!", style = MaterialTheme.typography.headlineSmall)
        Text("We're glad you're here.", style = MaterialTheme.typography.bodyMedium)
        Spacer(Modifier.height(16.dp))                         // extra space before the button
        Button(onClick = { }) { Text("Get started") }
    }
}
```

| Tool | Use |
|---|---|
| Spacer(Modifier.height(x.dp)) / width(x.dp) | Empty space between two specific elements |
| verticalArrangement = Arrangement.spacedBy(8.dp) | Equal space between **all** children of a Column |
| horizontalAlignment (Column) | Start, CenterHorizontally or End |
| verticalAlignment (Row) | Top, CenterVertically or Bottom |

> **Tip:** You'll learn every arrangement and alignment option, plus weight and Box, in **Week 5, Day 1**.

## 3. Surface: a background container

**Surface with colour and shape**

```
@Composable
fun TipBox(text: String) {
    Surface(
        color = MaterialTheme.colorScheme.secondaryContainer,
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    ) {
        Row(modifier = Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
            Icon(Icons.Default.Info, contentDescription = null)
            Spacer(Modifier.width(12.dp))
            Text(text)
        }
    }
}
```

A **Surface** automatically picks the matching text colour for its background (onSecondaryContainer here), so the text stays readable in both light and dark mode.

## 4. Card

**Card, ElevatedCard and OutlinedCard**

```
@Composable
fun CardExamples() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Card(modifier = Modifier.fillMaxWidth()) {
            Text("Card: filled background", modifier = Modifier.padding(16.dp))
        }
        ElevatedCard(modifier = Modifier.fillMaxWidth()) {
            Text("ElevatedCard: with a shadow", modifier = Modifier.padding(16.dp))
        }
        OutlinedCard(modifier = Modifier.fillMaxWidth()) {
            Text("OutlinedCard: with a border", modifier = Modifier.padding(16.dp))
        }
    }
}
```

## 5. Putting it together: a course card

**CourseCard**

```
@Composable
fun CourseCard(title: String, lessons: Int, level: String, modifier: Modifier = Modifier) {
    ElevatedCard(modifier = modifier.fillMaxWidth()) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(title, style = MaterialTheme.typography.titleLarge)
            Spacer(Modifier.height(4.dp))
            Text("$lessons lessons - $level", style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
            Spacer(Modifier.height(12.dp))
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Default.PlayArrow, contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary)
                Spacer(Modifier.width(4.dp))
                Text("Start learning", color = MaterialTheme.colorScheme.primary,
                    style = MaterialTheme.typography.labelLarge)
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun CourseCardPreview() {
    CourseCard(title = "Android with Kotlin", lessons = 96, level = "Beginner",
        modifier = Modifier.padding(16.dp))
}
```

## 6. Colours from the theme

| Colour token | Use |
|---|---|
| primary / onPrimary | Main brand colour / text on top of it |
| primaryContainer / onPrimaryContainer | Softer brand background / text on it |
| secondaryContainer, tertiaryContainer | Other soft backgrounds |
| surface / onSurface | Default background / normal text |
| onSurfaceVariant | Secondary text (subtitles) |
| error / onError | Errors |
| outline | Borders and lines |

**Careful:** Avoid fixed colours like `Color(0xFF000000)` in screens. Theme colours automatically work in dark mode; fixed colours don't.

## Practice Exercises

1. Build a **ProfileHeader**: a Row with a Person icon (56.dp), and next to it a Column with your name (titleLarge) and job title (bodyMedium).
2. Build a **StatCard(label, value)** using OutlinedCard, with the value in headlineMedium and the label in bodySmall.
3. Build a centred "Empty inbox" message: a big icon, a title and a description, with spacing between them.
4. Build a **NoticeBox(message)** using Surface with errorContainer colour and a Warning icon.

*Solutions are in the Answer Key at the end. Try first and look later!*

## Quick Quiz

**Q1.** What is the difference between Column and Row?

**Q2.** What does Arrangement.spacedBy(8.dp) do?

**Q3.** How do you centre children horizontally in a Column?

**Q4.** What does a Surface do automatically?

**Q5.** Name the three Material 3 card types.

WEEK 4 • DAY 6

# Weekly Project: Business Card App

**Today's goal:** Build a complete, good-looking Business Card app with everything from this week and run it on a device.

## 1. The design

**Sketch: What we're building**

```
+-----------------------------------+
|                                   |
|              ( AM )               |   <- circle with initials (or your photo)
|                                   |
|           Anjana Menon            |   <- headlineMedium, bold
|         Android Developer         |   <- titleMedium, primary colour
|                                   |
|  +-----------------------------+  |
|  | (phone)  +91 98470 11111    |  |
|  | --------------------------- |  |   <- Card with three ContactInfoRows
|  | (mail)   anjana@example.com |  |
|  | --------------------------- |  |
|  | (pin)    Kochi, Kerala      |  |
|  +-----------------------------+  |
+-----------------------------------+
```

## 2. Plan the composables

| Composable | Job | This week's skills used |
|---|---|---|
| BusinessCard | The whole screen | Column, alignment, background, Spacer |
| (circle Box) | Initials or photo | Box, size, clip(CircleShape), Text styles |
| ContactInfoRow | One icon + text line, reused 3 times | Row, Icon, modifier parameter |
| Card + HorizontalDivider | Groups the contact rows | Card, dividers |

**Tip:** Notice the plan: find the **repeated** part (the contact row) and make it its own composable with parameters. Then you write it once and use it three times.

## 3. The code

**BusinessCard.kt (create a new file: right-click your package → New → Kotlin Class/File → File)**

```
@Composable
fun BusinessCard(
    name: String,
    title: String,
    phone: String,
    email: String,
    location: String,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .background(MaterialTheme.colorScheme.primaryContainer)
            .padding(24.dp),
```

```
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // 1. Logo / photo area
        Box(
            modifier = Modifier
                .size(120.dp)
                .clip(CircleShape)
                .background(MaterialTheme.colorScheme.primary),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = name.split(" ").map { it.first() }.joinToString(""),
                style = MaterialTheme.typography.displaySmall,
                color = MaterialTheme.colorScheme.onPrimary
            )
        }
        Spacer(Modifier.height(16.dp))

        // 2. Name and title
        Text(
            text = name,
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.onPrimaryContainer
        )
        Text(
            text = title,
            style = MaterialTheme.typography.titleMedium,
            color = MaterialTheme.colorScheme.primary
        )
        Spacer(Modifier.height(32.dp))

        // 3. Contact details in a card
        Card(modifier = Modifier.fillMaxWidth()) {
            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                ContactInfoRow(icon = Icons.Default.Phone, text = phone)
                HorizontalDivider(modifier = Modifier.padding(horizontal = 16.dp))
                ContactInfoRow(icon = Icons.Default.Email, text = email)
                HorizontalDivider(modifier = Modifier.padding(horizontal = 16.dp))
                ContactInfoRow(icon = Icons.Default.LocationOn, text = location)
            }
        }
    }
}

@Composable
fun ContactInfoRow(icon: ImageVector, text: String, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.primary
        )
        Spacer(Modifier.width(16.dp))
        Text(text = text, style = MaterialTheme.typography.bodyLarge)
    }
```

```
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun BusinessCardPreview() {
    BusinessCard(
        name = "Anjana Menon",
        title = "Android Developer",
        phone = "+91 98470 11111",
        email = "anjana@example.com",
        location = "Kochi, Kerala"
    )
}
```

| Interesting line | What it does |
|---|---|
| name.split(" ").map { it.first() }.joinToString("") | "Anjana Menon" becomes "AM" (Week 2 collection operations!) |
| icon: ImageVector | Passing an icon as a parameter, so ContactInfoRow works for any icon |
| showSystemUi = true | The preview shows a whole phone frame |
| contentDescription = null | The icons are decorative, because the text next to them gives the meaning |

## 4. Show it in MainActivity

**MainActivity.kt (Android only)**

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Week4PracticeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    BusinessCard(
                        name = "Anjana Menon",
                        title = "Android Developer",
                        phone = "+91 98470 11111",
                        email = "anjana@example.com",
                        location = "Kochi, Kerala",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}
```

## 5. Use your own photo (optional)

**Replace the initials Box with an Image (Android only)**

```
Image(
    painter = painterResource(id = R.drawable.profile_photo),
    contentDescription = "Profile photo of $name",
    contentScale = ContentScale.Crop,
    modifier = Modifier
        .size(120.dp)
        .clip(CircleShape)
        .border(4.dp, MaterialTheme.colorScheme.primary, CircleShape)
)
```

## 6. Project challenges

- Add a fourth contact row for your **LinkedIn** or **GitHub** username (use Icons.Default.Share or AccountCircle).
- Make each ContactInfoRow **clickable**, and log which row was tapped with Log.d.
- Add a row of 3 skill chips (Kotlin, Compose, Git) under the job title, using the ClickableTag idea from Day 3.
- Run the app on your phone, take a screenshot, and push the project to GitHub (Week 3, Day 6).

## Week 4 checklist

| I can... | Done? |
|---|---|
| Create an Empty Activity project and run it on an emulator or phone | [ ] |
| Explain the main project files (MainActivity, res, Gradle, manifest) | [ ] |
| Use Log.d and Logcat | [ ] |
| Write composables with parameters and previews | [ ] |
| Style text with typography and colour tokens | [ ] |
| Use modifiers for size, padding, background, shape and clicks, in the right order | [ ] |
| Add buttons, icons and images | [ ] |
| Combine elements with Column, Row, Spacer, Surface and Card | [ ] |
| Build and run the Business Card app | [ ] |

## Quick Quiz

**Q1.** Why is ContactInfoRow a separate composable?

**Q2.** Why does BusinessCard accept a modifier parameter?

**Q3.** Why use MaterialTheme colours instead of fixed colours?

**Q4.** What does showSystemUi = true do in a preview?

**Q5.** What must you do before using R.drawable.profile_photo?

# Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

## Day 1: Android Studio & Your First App

**Quiz answers**

**Q1.** Empty Activity (the one with the Compose logo).

**Q2.** The build tool that compiles your code and resources into an APK.

**Q3.** res/drawable

**Q4.** The Android system, when the Activity is created.

**Q5.** Log.d(tag, message), and read them in Logcat.

## Day 2: Composable Functions & Text

**Exercise 1**

```
@Composable
fun MovieTitle(title: String, year: Int) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(title, style = MaterialTheme.typography.titleLarge)
        Text(year.toString(), style = MaterialTheme.typography.bodyMedium)
    }
}

@Preview(showBackground = true)
@Composable
fun MovieTitlePreview() {
    MovieTitle(title = "Drishyam", year = 2013)
}
```

**Exercise 2**

```
@Composable
fun QuoteCard(quote: String, author: String) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("\"$quote\"", fontStyle = FontStyle.Italic,
            style = MaterialTheme.typography.bodyLarge)
        Text("- $author", textAlign = TextAlign.End, modifier = Modifier.fillMaxWidth())
    }
}
```

**Exercise 3**

```
@Composable
fun WeatherInfo(city: String, temperature: Int) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("$city: $temperature C", style = MaterialTheme.typography.titleMedium)
        if (temperature > 35) {
            Text("Hot", color = MaterialTheme.colorScheme.error)
        } else {
            Text("Pleasant")
        }
    }
}
```

**Exercise 4**

```
@Composable
fun ShoppingList(items: List<String>) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Shopping list", style = MaterialTheme.typography.titleLarge)
        for (item in items) {
            Text("- $item")
        }
    }
}
```

**Quiz answers**

**Q1.** The function describes a piece of UI and can only be called from other composables.

**Q2.** Android Studio calls it by itself, so it passes sample data to the real composable.

**Q3.** sp

**Q4.** It keeps the app consistent, and the fonts can be changed in one place.

**Q5.** Yes. A composable is a normal Kotlin function.

## Day 3: Modifiers

**Exercise 1**

```
@Composable
fun RoundedSquare() {
    Box(
        modifier = Modifier
            .size(100.dp)
            .clip(RoundedCornerShape(16.dp))
            .background(MaterialTheme.colorScheme.primary)
            .border(2.dp, MaterialTheme.colorScheme.outline, RoundedCornerShape(16.dp))
    )
}
```

**Exercise 2**

```
// A and B from section 4; with clip after background, the background is drawn
// as a full rectangle first, so the clip no longer rounds the colour.
@Composable
fun ClipOrder() {
    Row(modifier = Modifier.padding(16.dp)) {
        Box(Modifier.size(64.dp).clip(RoundedCornerShape(16.dp))
            .background(MaterialTheme.colorScheme.primary))
        Spacer(Modifier.width(16.dp))
        Box(Modifier.size(64.dp).background(MaterialTheme.colorScheme.primary)
            .clip(RoundedCornerShape(16.dp)))
    }
}
```

**Exercise 3**

```
@Composable
fun Chip(label: String, selected: Boolean, onClick: () -> Unit, modifier: Modifier = Modifier) {
    val background = if (selected) MaterialTheme.colorScheme.primaryContainer
                     else MaterialTheme.colorScheme.surfaceVariant
    Text(
        text = label,
        modifier = modifier
            .clip(RoundedCornerShape(50))
            .background(background)
            .clickable { onClick() }
            .padding(horizontal = 16.dp, vertical = 8.dp)
    )
}
```

**Exercise 4**

```
@Composable
fun Banner(text: String) {
    Text(
        text = text,
        textAlign = TextAlign.Center,
        modifier = Modifier
            .fillMaxWidth()
            .background(MaterialTheme.colorScheme.tertiaryContainer)
            .padding(24.dp)
    )
}
```

**Quiz answers**

**Q1.** In the order they are written, from first to last.

**Q2.** fillMaxWidth takes all available width; width(200.dp) is an exact size.

**Q3.** So the ripple effect follows the rounded shape.

**Q4.** They flip automatically for right-to-left languages.

**Q5.** modifier: Modifier = Modifier, applied to the outermost element.

## Day 4: Buttons, Icons & Images

**Exercise 1**

```
@Composable
fun AuthButtons() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        Button(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Login") }
        OutlinedButton(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Sign up") }
        TextButton(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Continue as guest") }
        Button(onClick = { }, enabled = false, modifier = Modifier.fillMaxWidth()) { Text("Disabled") }
    }
}
```

**Exercise 2**

```
@Composable
fun PlayerControls() {
    Row {
        IconButton(onClick = { }) {
            Icon(Icons.AutoMirrored.Filled.KeyboardArrowLeft, contentDescription = "Previous")
        }
        IconButton(onClick = { }) {
            Icon(Icons.Default.PlayArrow, contentDescription = "Play")
        }
        IconButton(onClick = { }) {
            Icon(Icons.AutoMirrored.Filled.KeyboardArrowRight, contentDescription = "Next")
        }
        IconButton(onClick = { }) {
            Icon(Icons.Default.FavoriteBorder, contentDescription = "Add to favourites")
        }
    }
}
```

**Quiz answers**

**Q1.** So you can put any content inside it, such as an icon and text.

**Q2.** The filled Button.

**Q3.** For decorative icons or images whose meaning is already given by nearby text.

**Q4.** Lowercase letters, numbers and underscores only.

**Q5.** ContentScale.Crop

## Day 5: First Layouts, Surface & Card

**Exercise 1**

```
@Composable
fun ProfileHeader(name: String, job: String) {
    Row(modifier = Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
        Icon(Icons.Default.Person, contentDescription = null, modifier = Modifier.size(56.dp))
        Spacer(Modifier.width(12.dp))
        Column {
            Text(name, style = MaterialTheme.typography.titleLarge)
            Text(job, style = MaterialTheme.typography.bodyMedium)
        }
    }
}
```

**Exercise 2**

```
@Composable
fun StatCard(label: String, value: String, modifier: Modifier = Modifier) {
    OutlinedCard(modifier = modifier) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(value, style = MaterialTheme.typography.headlineMedium)
            Text(label, style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

**Exercise 3**

```
@Composable
fun EmptyInbox() {
    Column(
        modifier = Modifier.fillMaxSize().padding(32.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Icon(Icons.Default.Email, contentDescription = null, modifier = Modifier.size(72.dp),
            tint = MaterialTheme.colorScheme.outline)
        Spacer(Modifier.height(16.dp))
        Text("Empty inbox", style = MaterialTheme.typography.titleLarge)
        Spacer(Modifier.height(4.dp))
        Text("New messages will appear here.", textAlign = TextAlign.Center)
    }
}
```

**Exercise 4**

```
@Composable
fun NoticeBox(message: String) {
    Surface(
        color = MaterialTheme.colorScheme.errorContainer,
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    ) {
        Row(modifier = Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(12.dp))
            Text(message)
        }
    }
}
```

**Quiz answers**

**Q1.** Column places children vertically; Row places them horizontally.

**Q2.** It puts an equal 8.dp gap between all the children.

**Q3.** horizontalAlignment = Alignment.CenterHorizontally

**Q4.** It picks a readable content (text) colour that matches its background colour.

**Q5.** Card, ElevatedCard and OutlinedCard.

## Day 6: Weekly Project: Business Card App

**Quiz answers**

**Q1.** The same layout is used three times, so writing it once with parameters avoids repetition.

**Q2.** So the caller (MainActivity) can add things like innerPadding.

**Q3.** They match the design system and automatically adapt to dark mode.

**Q4.** It shows the composable inside a full phone frame.

**Q5.** Add an image called profile_photo to res/drawable.

**Next:** When you finish this week, tell your mentor and ask for the **Week 5** material. Great work, Anjana!